

Contents

Guide 5 — The Notification System, the simple version

1. The idea, in three sentences

2. How it works — five steps

3. The two streams

4. Where each part comes from (the research)

5. A worked example — one scrape cycle

6. The files

7. Jury questions — ready answers

1

1

1

2

2

2

3

Guide 5 — The Notification System, the simple version

How Mobile Match Algeria tells users and admins when something happens, in plain words, with the re-search behind it, a worked example, and ready answers for the jury.

1. The idea, in three sentences

When the catalogue changes, the right people should be told without having to refresh the page. The system keeps **two separate streams** — one for **users** (a new plan, a price drop, a fresh recommendation) and one for **admins** (a scrape finished or failed) — so each audience only sees what concerns them. Every alert is stored in the database and shown as a number on the **bell** in the navbar.

2. How it works — five steps

1. **Something happens** during a scrape run (a plan is added, a price falls, a scrape finishes...).

2. The scraper calls a small **notify** function for that event.

3. That function works out **who should be told** (which users, or the admins) and writes one **Notification** row per recipient.

4. The recipient's **bell** shows the count of unread alerts; opening it lists them.

5. Clicking an alert **routes** to the right page (the offer, or the admin scrape history), and it can be marked read or deleted.

3. The two streams

Stream	Types	Goes to	Fired when
User	new_offer	registered users (matching their preferences)	a scrape inserts a new plan
	price_drop	registered users (matching their preferences)	an existing plan's price falls
	recommendation	a user with a profile	the scrape's re-ranking finds them a strong match (score ≥ 75%)

Stream	Types	Goes to	Fired when
Admin	scrape_complete	all admins	a scrape run finishes successfully (a heartbeat, even with 0 new)
	scrape_failed	all admins	one or more operators failed to scrape

The two streams never cross: **admins are excluded from the user alerts**, and **users never receive the operational admin alerts**. This keeps each feed clean and relevant.

4. Where each part comes from (the research)

Part	What it does	Comes from
Two separate streams	each audience gets only its events	the publish / subscribe pattern, where producers post events to topics and only interested subscribers receive them (Eugster et al., 2003)
Bell + unread badge, click to act	inform without interrupting the task	notification design research on the attention / utility trade-off (McCrickard & Chewar, 2003)

What is *ours* is the configuration: which events exist, who subscribes to each, the preference filter on user alerts, and the 75% threshold for a recommendation alert.

5. A worked example — one scrape cycle

An admin clicks **Run scrape**. During the run:

1. A new Djazzy plan is inserted → every user whose preferences fit gets a `new_offer` alert (“New offer from Djazzy!”).
2. A Mobilis plan drops from 1,500 to 1,300 DA → matching users get a `price_drop` alert (“...dropped from 1500 DA to 1300 DA (−13%)”).
3. The engine re-ranks every profile; one user’s best match scores 88% → that user gets a `recommendation` alert (“Match found! (88%)”).
4. The run finishes successfully → every admin gets a `scrape_complete` alert (“Scrape complete — no new offers · Djazzy 33 · Ooredoo 37 · Mobilis 13”).
5. If an operator had failed, every admin would instead also get a `scrape_failed` alert with the error.

Each alert appears as a +1 on that person’s bell; clicking a plan alert opens the plan, clicking an admin alert opens the scrape history.

6. The files

- [scraper/index.ts](#) — the `notify...` functions that create the alerts during a scrape (`notifyUsersNewOffer`, `notifyUsersPriceDrop`, `notifyPersonalizedRecommendations`, `notifyAdmins`, `notifyAdminsFailure`).
 - [api/notifications/route.ts](#) — list (GET), mark read (PATCH), delete (DELETE), scoped to the logged-in user.
 - [components/navbar.tsx](#) — the bell, the unread badge, and the click-through routing.
 - **Prisma Notification model** — stores `userId`, `title`, `message`, `type`, `isRead`, `offerId`.
-

7. Jury questions — ready answers

- **Why two separate streams?** So each audience sees only what concerns it: a shopper does not want scrape logs, and an admin does not want a “match found” alert. It is the publish/subscribe idea — events go only to their subscribers.
 - **Is it real-time / push?** No. It is database-backed: alerts are written during the scrape and the bell shows them on the next page load. That is enough for a daily catalogue and avoids the complexity of websockets.
 - **Why do some users not get a `new_offer`?** User alerts respect the profile preferences (preferred plan type and network), so people are not spammed with irrelevant plans.
 - **Why a 75% threshold on recommendations?** To alert only on a genuinely strong match, not on every minor re-ranking.
 - **Can a user read another user’s notifications?** No. Every notification has a `userId`, and the API only ever returns or modifies rows for the logged-in user (verified by the multi-account isolation test in Chapter 4).
 - **What stops the daily cron from spamming admins?** One `scrape_complete` per run (once a day from the cron), and failures are summarised into a single alert.
-

Next guide: comparison and savings, or responsive design and accessibility. Tell me which.